

Mixins

Problem

JS me ek class sirf **ek** class ko extend kar sakti hai (ek hi `[[Prototype]]`). Lekin kabhi do jagah se features chahiye hote hain, jaise `User` me `EventEmitter` ki power bhi.

Mixin kya hai?

Ek aisa object/class jisme kuch methods hote hain, jo doosri classes me **jod** diye jaate hain, bina inherit kiye. Mixin akela use nahi hota, wo bas behavior "add" karta hai.

Simple example

Methods ko class ke prototype me copy kar do (`Object.assign`).

javascript

```
let sayHiMixin = {  
  sayHi() { console.log(`Hello ${this.name}`); },  
  sayBye() { console.log(`Bye ${this.name}`); }  
};
```

```
class User {  
  constructor(name) { this.name = name; }  
}
```

```
Object.assign(User.prototype, sayHiMixin);
```

```
new User("Dude").sayHi(); // Hello Dude
```

Ye inheritance nahi, sirf **methods ki copy** hai. To `User` kisi aur class ko extend bhi kar sakta hai aur mixin bhi le sakta hai:

javascript

```
class User extends Person {}  
Object.assign(User.prototype, sayHiMixin);
```

Mixin ke andar inheritance

Mixin doosre mixin se inherit kar sakta hai (`__proto__`), aur `super` bhi chalta hai.

javascript

```
let sayMixin = { say(phrase) { console.log(phrase); } };

let sayHiMixin = {
  __proto__: sayMixin,
  sayHi() { super.say(`Hello ${this.name}`); }
};
```

`super` yahan **mixin ke prototype** me dhoondhta hai, class ke nahi. Wajah: methods ka `[[HomeObject]]` mixin hi rehta hai, copy hone ke baad bhi.

EventMixin (real-life example)

Kisi bhi class ko events dene ke liye:

- `.on(name, handler)`: event sunne ke liye handler jodo
- `.off(name, handler)`: handler hatao
- `.trigger(name, ...args)`: event generate karo, saare handlers chalte hain

javascript

```
class Menu {
  choose(value) { this.trigger("select", value); }
}
Object.assign(Menu.prototype, eventMixin);

let menu = new Menu();
menu.on("select", value => console.log(`Selected: ${value}`));
menu.choose("123"); // Selected: 123
```

Andar handlers `_eventHandlers` object me array ki tarah store hote hain (event ke naam ke hisaab se).

Dhyan rakho

- Mixin galti se class ke existing method ko **overwrite** kar sakta hai. Isliye mixin ke method naam soch samajh kar rakho.

Yaad rakho

- JS me multiple inheritance nahi hai, mixin uska practical tareeka hai.
- `Object.assign(Class.prototype, mixin)` se behavior jodo.
- Ek class kai mixins le sakti hai, inheritance chain bina chhede.